

debugging proof template

student: ezlan sohani

assessment / task: parking lot tracker - part a

date: 2026-09-12

bug 1 - empty lot totals wrong

1. what was the problem?

expected behaviour: total bays 3, occupied bays 0, free bays 3

actual behaviour: total bays 3, occupied bays 1, free bays 2

2. evidence of the bug

terminal screenshot 1: the empty lot output showed occupied bays 1 and free bays 2.

3. what caused the problem?

relevant code: `i_occupied = 1`

explanation: the counter started at one, so the empty lot was counted as having one occupied bay. it should start at zero.

4. how did you debug it?

tested `calculateTotals()` with three free bays and compared expected and actual output.

5. what did you change?

corrected code: `i_occupied = 0`

the counter now starts at zero before counting occupied bays.

6. proof that the fix works

test performed: empty car park totals

expected result: total 3, occupied 0, free 3

actual result after fix: total 3, occupied 0, free 3

result: pass

evidence: corrected terminal screenshot supplied by the student.

bug 2 - removed bay stays occupied

1. what was the problem?

expected behaviour: plate cleared and bay 1 marked free

actual behaviour: plate cleared but bay 1 remained marked occupied

2. evidence of the bug

terminal screenshot 2: bay 1 showed occupied after the car was removed.

3. what caused the problem?

relevant code: `dictBays[bay_number]["occupied"] = True`

explanation: the removal code cleared the plate but assigned true to occupied. the bay should be marked false.

4. how did you debug it?

tested entry, removal, and status display, then compared the expected free state with the actual state.

5. what did you change?

corrected code: `dictBays[bay_number]["occupied"] = False`

a removed car now leaves its bay free.

6. proof that the fix works

test performed: remove the car from bay 1 and view status

expected result: bay 1 is free and its plate is cleared

actual result after fix: bay 1 is free and its plate is cleared

result: pass

debugging proof template

evidence: corrected terminal screenshot supplied by the student.

bug 3 - saved occupied bay reloads as free

1. what was the problem?

expected behaviour: bay 1 stays occupied with plate abc123 after save and reload

actual behaviour: bay 1 reloaded as free

2. evidence of the bug

parking_data.txt contained 1|abc123|True but status showed bay 1 free after restart.

3. what caused the problem?

relevant code: `occupied_text == "true"`

explanation: `loadData()` compared `occupied_text` with lowercase `true`, but `saveData()` wrote python `True` with capital `t`.

4. how did you debug it?

compared the value written to `parking_data.txt` with the value checked by `loadData()`.

5. what did you change?

corrected code: `dictBays[bay_number]["occupied"] = occupied_text == "True"`

the load check now matches the saved value.

6. proof that the fix works

test performed: saved an occupied bay, closed the program, and loaded the data again

expected result: bay 1 is occupied with plate abc123

actual result after fix: bay 1 is occupied with plate abc123

result: pass

evidence: corrected reload terminal screenshot supplied by the student.

final debugging reflection

which bug was the most difficult to solve, and why?

the reload bug was most difficult because saving and loading used different boolean text values.

what debugging technique was most useful?

comparing saved data with the loading condition was most useful.

what did you learn that could help you avoid or identify similar bugs in future?

i learned to test empty, occupied, removed, and reloaded states and to check both sides of file handling.